

Documento de Gerenciamento de Configuração de Software (GCS)

Projeto: Site Achei

Este documento estabelece as políticas de desenvolvimento, padrões de código e gerência de mudanças para o projeto **Site Achei**. O cumprimento destas normas é obrigatório para todos os membros da equipe.

1. Organização e Gestão de Tarefas (Issues e Projects)

Para evitar duplicidade de trabalho e garantir que todos saibam o que está acontecendo no projeto, utilizamos o ecossistema de gestão do GitHub.

O Quadro Kanban (GitHub Projects):

O progresso do desenvolvimento é acompanhado visualmente na aba **Projects** do repositório, dividida em três colunas principais:

- **Todo (A Fazer):** Issues abertas que aguardam um desenvolvedor.
- **In Progress (Em Andamento):** Tarefas que já estão sendo codificadas.
- **Done (Concluído):** Tarefas finalizadas e integradas.

Ciclo de uma Tarefa (Issue):

1. **Registro:** Qualquer nova funcionalidade, bug ou melhoria deve ser registrado abrindo uma **Issue** descritiva.
2. **Atribuição e Label:** A Issue deve receber uma etiqueta (ex: bug, enhancement, documentation) e ser atribuída (*Assignee*) ao membro da equipe que executará o trabalho.
3. **Movimentação:** Ao iniciar o desenvolvimento, o responsável deve mover a Issue para a coluna **In Progress** no quadro do projeto.

2. Fluxo de Trabalho (GitHub Flow)

Adotamos o **GitHub Flow** devido à sua simplicidade e foco em entrega contínua. A regra de ouro é: **tudo o que está na branch main deve estar estável e pronto para produção.**

Ciclo de Desenvolvimento:

Sincronize seu ambiente local: Antes de iniciar qualquer tarefa, atualize sua branch principal.

None

Bash

```
git checkout main  
git pull origin main
```

1. **Crie uma branch referenciando a Issue:** Crie a ramificação a partir da main. Inclua o número da Issue no nome da branch para facilitar a rastreabilidade.
 - *Nomenclatura:* feature/issue-[Nº]-[nome-curto] ou bugfix/issue-[Nº]-[nome-curto].

Shell

Exemplo para a Issue #12

```
git checkout -b feature/issue-12-controle-estoque
```

2. **Desenvolva e Commite:** Faça as alterações necessárias e realize os commits seguindo o padrão de mensagens (ver Seção 2).

Abra um Pull Request (PR): Envie sua branch para o GitHub e abra um PR direcionado para a branch main.

None

Bash

```
git push origin feature/controle-estoque
```

3. **Revisão e Merge:** Pelo menos um outro membro da equipe deve revisar o código no GitHub. Após a aprovação e validação, o merge na main é realizado e a branch remota pode ser deletada.

3. Padronização de Commits (Conventional Commits)

As mensagens de commit devem ser claras e estruturadas para manter o histórico do projeto legível e organizado.

Formato do Commit:

Textproto

```
<tipo>(<escopo opcional>): <descrição curta em letras minúsculas>
```

Tipos Permitidos:

- **feat:** Introdução de uma nova funcionalidade (ex: feat(medicamentos): adiciona listagem de estoque).
- **fix:** Correção de um bug (ex: fix(auth): corrige expiração do token de acesso).
- **docs:** Alterações exclusivas na documentação (ex: docs: atualiza instruções de instalação no readme).
- **style:** Mudanças de formatação que não alteram o comportamento do código (espaços, recuo, ponto e vírgula, etc.).
- **refactor:** Reestruturações de código que não alteram funcionalidade nem corrigem bugs.
- **chore:** Atualizações de tarefas de build, pacotes, ou ferramentas de configuração (ex: chore: adiciona dependência no requirements.txt).

4. Política de Resolução de Conflitos

Os conflitos devem ser **obrigatoriamente resolvidos no ambiente local do desenvolvedor** antes de tentar concluir o Pull Request no GitHub.

Passo a Passo para Resolução:

Enquanto estiver na sua branch de tarefa, puxe as atualizações mais recentes da main:

None

Bash

```
git fetch origin
```

```
git merge origin/main
```

1. O Git identificará os arquivos conflitantes. Abra o editor de código (ex: VS Code) para avaliar as linhas marcadas.
 2. Escolha a alteração correta (manter a sua, manter a da main ou combinar ambas).
- Salve os arquivos, adicione-os ao stage e conclua o processo enviando a resolução para o repositório:

None

Bash

```
git add .  
git commit -m "chore: resolve conflitos de integração com a  
main"  
git push origin feature/sua-feature
```